

SHIDEN 紫電

Distributed In-Memory Data Grid

PoC 1 Consistent Hashing Ring

Building the foundation for scalable data distribution.



Uniform Key Distribution



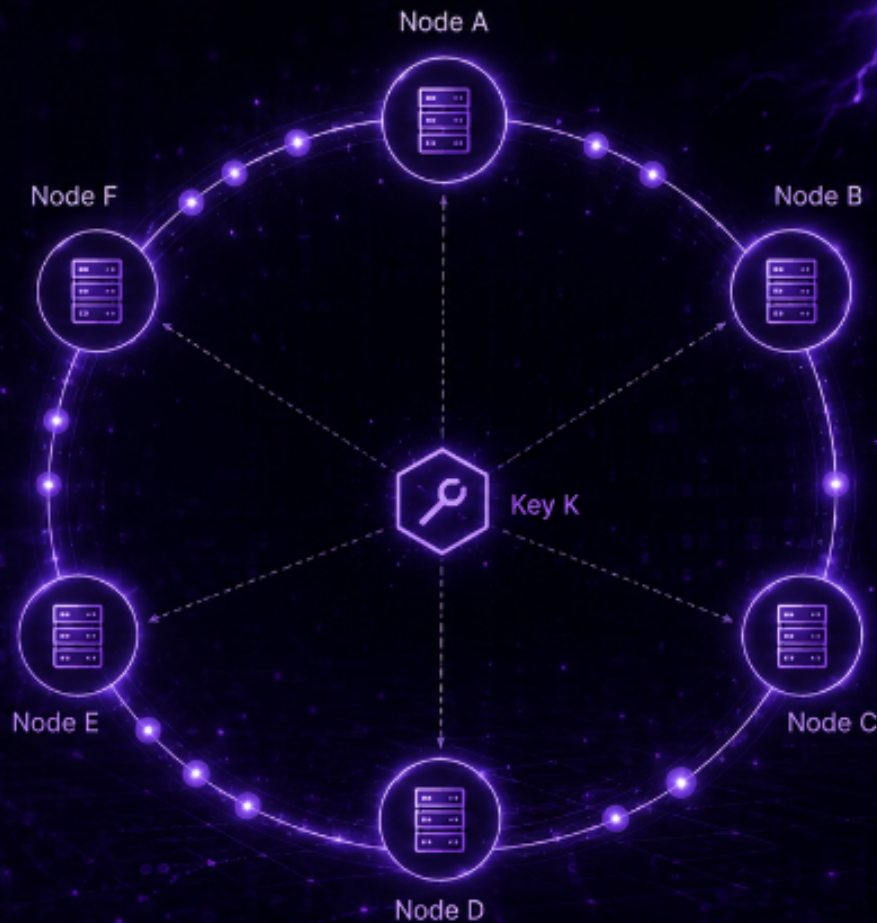
Minimal Data Movement



Virtual Nodes Support



Fault Tolerant by Design



// BUILDING SHIDEN
RFC-005 in Progress



POC

01 // TOPOLOGY

Consistent Hash Ring

Modulo-based hashing ($hash(key) \pmod N$) fails in elastic environments because scaling the cluster size N invalidates almost all cache mappings.

Consistent hashing maps both **servers** and **data keys** onto a circular $2^{32} - 1$ integer ring. A key routes clockwise to the first virtual node (VNode) it encounters.

To achieve uniform key distribution, we scatter multiple VNodes per physical node across the ring:

$$VNode_{address} = \text{Murmur3}(\text{node_id} + "\#" + i)$$

Takeaway: VNode scattering smooths hash space allocation, reducing load variation across nodes to a Coefficient of Variation (CV) of **2.98%**.

IMPLEMENTATION & CONFIGURATION

```
// VNode registration: Map servers to ring
for (int i = 0; i < vnodes; i++) {
    int hash = murmur3(node.id() + "#" + i);
    ring.put(hash, node);
}

// Clockwise route lookup on sorted array
int idx = Arrays.binarySearch(hashes, keyHash);
if (idx < 0) {
    idx = -idx - 1;
    if (idx >= hashes.length) idx = 0; // Wrap
}
return nodes[idx];
```

Simulation Topology Parameters:

- **Physical Nodes:** 5 (Node-1 to Node-5)
- **VNodes per Node:** 150 (Total of 750 tokens)
- **Consistent Hashing CV:** **2.98%** (Uniform balance)
- **Key Count:** 100,000 simulated keys

02 // HASH COLLAPSE

JDK hashCode() Defect

Standard JDK `String.hashCode()` is a polynomial rolling hash:

$$h = \sum_{i=0}^{n-1} s[i] \cdot 31^{n-1-i}$$

Under sequential vnode naming patterns (e.g. "node-X#Y"), the resulting hash codes cluster inside a tiny **0.00009%** range of the 32-bit integer space.

Because the server boundaries fail to scatter across the ring, nearly all simulated keys map outside the clustered range, routing clockwise to the first active node.

Defect Result: Under our sequential vnode naming scheme, `String.hashCode()` produced severe clustering, routing 100% of keys to Node-1 and exploding load variation CV to **200.00%**.

DISTRIBUTION METRICS & COLLAPSE TEST

Metric	Murmur3 (32-bit)	JDK hashCode()
Node-1 Keys	20,432	100,000
Node-2 Keys	19,411	0
Node-3 Keys	20,103	0
Node-4 Keys	19,951	0
Node-5 Keys	20,103	0
CV %	2.98%	200.00%

```
// JDK hashCode clustering demonstration
for (int i = 0; i < 150; i++) {
    String vnode = "node-1#" + i;
    // Hashes end up clustered together
    int code = vnode.hashCode();
}
```

03 // REBALANCING

Scale-up Key Migration

Adding a node to the cluster requires data migration. Modulo hashing forces $N/(N + 1) = 83.33\%$ of keys to redistribute. Consistent hashing minimizes logical key movement to the optimal 14.56%.

However, Consistent Hashing (Strategy A) splits keys into $N \times V = 900$ small token range boundaries. This forces the storage engine to perform hundreds of random disk reads and split-compaction writes.

Takeaway: Decoupling the data movement layout from the token ranges is required to prevent random disk I/O bottlenecks.

REBALANCE SIMULATION (10 GB DATASET)

Strategy	Keys Migrated	I/O Throughput	Est. Time
Naive Modulo	83.49%	100 MB/s	85.5 seconds
Consistent Ring	14.56%	15 MB/s	99.4 seconds
Slot Hashing	13.34%	120 MB/s	11.4 seconds

Environment: NVMe SSD, RF=1, sequential slot files, Linux ext4.

Physical I/O Detail:

Modulo relocates 83.49% of data. Consistent hashing minimizes logical key movement to 14.56% but thrashes storage seeks on 900 ranges. Slot Hashing streams contiguous partition files sequentially at 120+ MB/s, cutting rebalance to 11s.

04 // OPTIMIZATION

Fixed Slot Partitioning

Fixed Slot Partitioning introduces an indirection layer that decouples logical routing from physical storage nodes. We map a fixed number of partitions (e.g. $Q = 1024$ slots) to servers.

Client routing is a flat $O(1)$ bitwise mask array lookup.

During scale-up, we compare the slot allocation table to find migrated slots (taking sub-millisecond times), and stream their contiguous slot directories directly, eliminating key-by-key database scans.

Takeaway: Fixed Slot Partitioning achieves flat **28.8 ns** lookup times and speeds up database rebalancing by **9x** over Consistent Hashing.

PERFORMANCE BENCHMARKS

```
// Slot Table lookup: O(1) route logic
public Node route(String key) {
    int hash = murmur3(key);
    int slot = hash & (numSlots - 1);
    return slotTable[slot];
}
```

JMH Lookup Latency Results:

- **Slot Hashing lookup: 28.8 ns/op** ($O(1)$ flat lookup)
- **Consistent Ring (V=500): 164.1 ns/op** ($O(\log NV)$ search)
- **Topology Diffing (Q=1024): 0.30 ms** (Slots) vs. **21.17 ms** (Key-scanning)